Enterprise Data Observability Accelerator

Technical Reference & Deployment Guide

Microsoft Fabric · PySpark · Delta Lake · Power BI - May 2026

Arghyadeep Paul | Data Analytics & AI | Embee Software Pvt Ltd.

1. What This Accelerator Does

The EDO Accelerator is a metadata-driven, config-first data quality engine built on Microsoft Fabric. It ingests any enterprise Excel or tabular dataset into a Lakehouse Bronze table, applies a configurable set of validation rules against it, computes an AI-powered severity score, and writes a timestamped observability log to a Gold table that can be consumed directly by Power BI.

The engine is universal by design: the validation logic, severity scoring, and Gold layer write are static. Only two inputs change per deployment - the dataset name and the rules configuration. This makes it a reusable template solution, not a one-off ETL script.

Capability	How It Works
Dataset ingestion	Reads any .xlsx from Lakehouse Files; auto-normalises column names; writes to a Bronze Delta table named from the dataset
Validation engine	Dynamically reads rules from validation_rules config table; supports null_check, negative_check, duplicate_check, accepted_values, freshness_check
AI severity scoring	Calculates fail percentage across all rules; classifies as CRITICAL / HIGH / MEDIUM / HEALTHY
Observability log	Appends results to gold_observability_log with target_table, severity, and evaluated_at timestamp
Multi-dataset support	Run the same notebook on any dataset by changing one variable; all runs accumulate in the Gold log

2. Architecture

The accelerator follows a Bronze-to-Gold medallion pattern with a dedicated observability layer rather than a business transformation layer.

Layer	Table Name	Contents
Bronze	bronze_{dataset_name}	Raw ingested data; immutable; Delta format

Layer	Table Name	Contents
Config	validation_rules	Rule definitions: column, check type, threshold
Gold	gold_observability_log	Validation results, severity scores, run timestamps

Data flow: Excel file → Lakehouse Files → Bronze table → Validation engine reads rules → PASS/FAIL per rule → Severity score → Gold observability log → Power BI.

3. Notebook Structure

The notebook has 8 cells. Cells 1 and 3 are the only cells that change between deployments. Cells 2 and 4–8 are permanently frozen.

Cell	Purpose	Changes per deployment?
Cell 1	Set DATASET_NAME variable	Yes — one word only
Cell 2	Ingest Excel into Bronze table	Never
Cell 3	Define validation rules for this dataset	Yes — column names and thresholds
Cell 4	Load Bronze table and rules into memory	Never
Cell 5	Run validation engine against all rules	Never
Cell 6	Compute AI severity score	Never
Cell 7	Write results to Gold observability log	Never
Cell 8	Display full Gold log (demo view)	Never

Cell 1 — Config (the only identity of a deployment)

```
DATASET_NAME = "Retail_Observability_Intelligence"   # ← change this only
```

Cell 2 auto-derives the file path and Bronze table name from this variable. No other cell references it directly.

Cell 3 — Validation Rules

Rules are defined as a Python list and written to the validation_rules Lakehouse table. The rule schema is fixed; only the values change.

Field	Type	Description
rule_id	String	Unique identifier (R001, R002 …)
column_name	String	Exact column name in the Bronze table (case-insensitive match)
validation_type	Enum	null_check | negative_check | duplicate_check | accepted_values | freshness_check
threshold	String	Numeric threshold, list of accepted values, or hours for freshness checks

4. Validation Rule Types

Validation Type	What It Checks	Threshold Format	Example
null_check	Percentage of null/blank values in column	Numeric (max % allowed)	"0" = zero nulls permitted
negative_check	Count of values below zero	Numeric (always 0)	"0" = no negatives permitted
duplicate_check	Percentage of duplicate values in column	Numeric (max % allowed)	"0.1" = up to 0.1% duplicates OK
accepted_values	Values not in the approved set	Python list as string	"['APAC','EMEA','NA','LATAM']"
freshness_check	Age of the most recent record	Hours as string	"200 hrs" = fail if latest > 200 hours old

5. Severity Scoring Logic

After all rules are evaluated, the engine calculates a single severity score for the entire dataset run based on the percentage of rules that failed.

Severity	Condition	Meaning
CRITICAL	50% or more of rules failed	Immediate attention required before data is used
HIGH	25–49% of rules failed	Significant issues; investigate within hours
MEDIUM	1–24% of rules failed	Minor issues; schedule a fix
HEALTHY	0% of rules failed	All checks passed; data is trustworthy

6. Datasets Validated

The following datasets have been run through the accelerator. All three were processed by the same notebook with only Cell 1 and Cell 3 changed between runs.

Dataset	Bronze Table	Rules Applied	Score	Key Findings
sales_pipeline_data.xlsx	bronze_sales_pipeline_data	5	MEDIUM	8 out-of-range region values
Retail_Observability_Intelligence.xlsx	bronze_retail_observability_intelligence	5	CRITICAL	11% null customer IDs; 200 invalid region values; data 607 hours old
bronze_hr (synthetic)	bronze_hr	5	CRITICAL	Duplicate IDs; null customer; negative deal value; invalid region

7. How to Deploy on a New Dataset

Follow these steps each time the accelerator is pointed at a new dataset.

Step	Action	Where
1	Upload the dataset Excel file to the Lakehouse Files folder	Fabric Lakehouse → Files
2	Open the accelerator notebook (AI_Sales_Accelerator)	Fabric Notebook
3	In Cell 1, set DATASET_NAME to the exact filename without .xlsx	Cell 1 only
4	In Cell 3, update the rules list to match the new dataset's column names	Cell 3 only
5	Run All Cells (or run cells 1–8 in order)	Notebook toolbar
6	Check Cell 6 output for the severity score	Cell 6 output
7	Check Cell 8 to see the full Gold log with all historical runs	Cell 8 output

Cells 2 and 4–8 require no changes. They read from the variables and tables set in Cells 1 and 3 automatically.

8. Gold Observability Log Reference

Every notebook run appends rows to gold_observability_log. This table is the single source of truth for data quality history and the basis for any Power BI dashboard.

Column	Type	Description
rule_id	String	The rule identifier from the config table
column_name	String	The column that was validated
validation_type	String	The check type applied
threshold	String	The configured threshold
status	String	PASS, FAIL, or SKIPPED (column not found in dataset)
detail	String	Quantified finding (e.g. '11.0% nulls', '8 out-of-range values')
target_table	String	The Bronze table that was validated
severity	String	The overall run severity: CRITICAL / HIGH / MEDIUM / HEALTHY
evaluated_at	Datetime	Timestamp of the run (ISO 8601)

9. Fabric Components

Component	Name	Role
Workspace	Sales_Intelligence	Container for all accelerator assets
Lakehouse	Sales_Intelligence_LH	Stores Bronze tables, validation_rules config, and Gold observability log
Notebook	AI_Sales_Accelerator	The accelerator engine; 8 cells; Cells 1 and 3 configurable
Gold Table	gold_observability_log	Append-only run history; source for Power BI
Config Table	validation_rules	Rule definitions; updated per dataset; no notebook changes required

10. Quick Reference

Task	Action
Point accelerator at a new dataset	Change DATASET_NAME in Cell 1 and update column rules in Cell 3

Task	Action
Add a new validation rule	Add a row to validation_rules table (or update Cell 3 and re-run)
View all historical runs	Run Cell 8: display(spark.table('gold_observability_log'))
Clear test history before demo	spark.sql('TRUNCATE TABLE gold_observability_log')
Connect to Power BI	Create semantic model pointing at gold_observability_log in the Lakehouse
Run on a second dataset in same session	Only re-run Cells 1, 3–8; Cell 2 re-ingests from file